

Verify the out-of-fold weight guard test actually has teeth

Context

`tests/test_ensemble.py::test_oof_weights_ignore_the_fold_being_scored` was found to be vacuous. It called `fit_oof_weights(earlier, earlier_actual, settings)` twice with identical arguments and compared the results, so it could never detect the thing it was named after — the actual scored fold was never passed in at all.

This was proven by mutation testing: replacing the entire body of `fit_oof_weights` with a constant that returned uniform weights left the test passing.

The underlying behavior was separately verified as correct. The ensemble genuinely does fit weights only from earlier scored folds — `evaluation/backtest.py` fits at line 849 from `prior_tests`, scores at 881, and appends at 922 to 923, so no fold's weights ever see its own rows. The bug was in the test, not the code. A cleanup pass has since been run against this.

This card verifies that the cleanup actually fixed it. It is a verification card, not a fix card.

Tasks

1. Report the current body of `test_oof_weights_ignore_the_fold_being_scored` in full. Paste the code as it stands now.
2. State plainly whether the test now passes the scored fold's own data into the call it is asserting against. If it still does not, say so and stop — report it as unfixed rather than repairing it inside this card.
3. Mutation-test it. Replace the body of `fit_oof_weights` with a constant that ignores its inputs and returns uniform weights across members. Run only this test. Paste the output.
4. Report the mutation result honestly. If the test still passes under the mutation, it is still vacuous regardless of how it reads. That is the finding, and it should be reported as a failure of the cleanup pass.
5. Restore `fit_oof_weights` to its original implementation and confirm the full test suite returns to its prior state. Report pass, fail, and skip counts, and confirm no mutation code remains anywhere in the working tree.
6. Apply the same mutation check to any other test added or changed by the cleanup pass that claims to guard against lookahead or data leakage. For each, report the test name and whether it survived mutation.

Do not

Do not leave mutated code in the repository. Do not modify the test in this card, even if it is still vacuous — report it. Do not change `evaluation/backtest.py`; its accumulation order was verified correct and is not in question here.

Report

Report which items you completed and explicitly list any you did not do and why. Do not silently skip items. Paste actual command output for items 3 and 5 rather than describing what happened.

What to look for

The question this card asks is not "does the test pass." It is "would this test fail if the behavior were broken." Those are different questions and only the second one is worth anything.

This repository has a documented pattern of producing tests that read as though they check something without actually checking it. Mutation testing — deliberately breaking the code and confirming the test notices — is the only cheap way to tell the difference. Item 6 extends the check to the rest of the cleanup pass, because a pattern that appeared once in a leakage guard is likely to appear again in the others.